

Submdspan

Document #: P2630
Date: 2022-10-14
Project: Programming Language C++
LEWG
Reply-to: Christian Trott
<crtrott@sandia.gov>
Damien Lebrun-Grandie
<lebrungrandt@ornl.gov>
Mark Hoemmen
<mhoemmen@nvidia.com>

Contents

- [1 Revision History](#)
 - [1.1 Revision 1: Mailing 2022-10](#)
 - [1.2 Initial Version 2022-08 Mailing](#)
- [2 Description](#)
 - [2.1 Design of submdspan](#)
 - [2.1.1 Slice Specifiers](#)
 - [2.1.2 Customization Points](#)
 - [2.1.3 Making sure submdspan behavior meets expectations](#)
- [3 Wording](#)
 - [3.1 Add subsection 22.7.X \[mdspan.submdspan\] with the following](#)

§ 1 Revision History

§ 1.1 Revision 1: Mailing 2022-10

- updated rational for introducing `strided_index_range`
- merged `submdspan_mapping` and `submdspan_offset` into a single customization point `submdspan_mapping` returning a pair of the new mapping and the offset.
- update wording for argument dependent lookup.

§ 1.2 Initial Version 2022-08 Mailing

§ 2 Description

Until one of the last revisions, the `mdspan` paper P0009 contained `submdspan`, the `subspan` or “slicing” function that returns a view of a subset of an existing `mdspan`. This function was considered critical for the overall functionality of `mdspan`. However, due to review time constraints it was removed from P0009 in order for `mdspan` to be included in C++23.

This paper restores `submdspan`. It also expands on the original proposal by

- defining customization points so that `submdspan` can work with user-defined layout policies, and
- adding the ability to specify slices as compile-time values.

Creating subspans is an integral capability of many, if not all programming languages with multidimensional arrays. These include Fortran, Matlab, Python, and Python’s NumPy extension.

Subspans are important because they enable code reuse. For example, the inner loop in a dense matrix-vector product actually represents a *dot product* – an inner product of two vectors. If one already has a function for such an inner product, then a natural implementation would simply reuse that function. The LAPACK linear algebra library depends on subspan reuse for the performance of its one-sided “blocked” matrix factorizations (Cholesky, LU, and QR). These factorizations reuse textbook non-blocked algorithms by calling them on groups of contiguous columns at a time. This lets LAPACK spend as much time in dense matrix-matrix multiply (or algorithms with analogous performance) as possible.

The following example demonstrates this code reuse feature of subspans. Given a rank-3 `mdspan` representing a three-dimensional grid of regularly spaced points in a rectangular prism, the function `zero_surface` sets all elements on the surface of the 3-dimensional shape to zero. It does so by reusing a function `zero_2d` that takes a rank-2 `mdspan`.

```
// Set all elements of a rank-2 mdspan to zero.
template<class T, class E, class L, class A>
void zero_2d(mdspan<T,E,L,A> grid2d) {
    static_assert(grid2d.rank() == 2);
    for(int i = 0; i < grid2d.extent(0); ++i) {
        for(int j = 0; j < grid2d.extent(1); ++j) {
            grid2d[i,j] = 0;
        }
    }
}

template<class T, class E, class L, class A>
void zero_surface(mdspan<T,E,L,A> grid3d) {
    static_assert(grid3d.rank() == 3);
    zero_2d(submdspan(grid3d, 0, full_extent, full_extent));
    zero_2d(submdspan(grid3d, full_extent, 0, full_extent));
    zero_2d(submdspan(grid3d, full_extent, full_extent, 0));
}
```

```

    zero_2d(submdspan(grid3d, grid3d.extent(0)-1, full_extent,
        full_extent));
    zero_2d(submdspan(grid3d, full_extent, grid3d.extent(1)-1,
        full_extent));
    zero_2d(submdspan(grid3d, full_extent, full_extent,
        grid3d.extent(2)-1));
}

```

§ 2.1 Design of submdspan

As previously proposed in an earlier revision of P0009, `submdspan` is a free function. Its first parameter is an `mdspan` `x`, and the remaining `x.rank()` parameters are slice specifiers, one for each dimension of `x`. The slice specifiers describe which elements of the range `[0,x.extent(d))` are part of the multidimensional index space of the returned `mdspan`.

This leads to the following fundamental signature:

```

template<class T, class E, class L, class A,
        class ... SliceArgs>
auto submdspan(mdspan<T,E,L,A> x, SliceArgs ... args);

```

where `E.rank()` must be equal to `sizeof...(SliceArgs)`.

§ 2.1.1 Slice Specifiers

§ 2.1.1.1 P0009's slice specifiers

In P0009 we originally proposed three kinds of slice specifiers.

- A single integral value. For each integral slice specifier given to `submdspan`, the rank of the resulting `mdspan` is one less than the rank of the input `mdspan`. The resulting multidimensional index space contains only elements of the original index space, where the particular index matches this slice specifier.
- Anything convertible to a `tuple<mdspan::index_type, mdspan::index_type>`. The resulting multidimensional index space covers the begin-to-end subrange of elements in the original index space described by the `tuple`'s two values.
- An instance of the tag class `full_extent_t`. This includes the full range of indices in that extent in the returned subspan.

§ 2.1.1.2 Strided index range slice specifier

In other languages which provide multi dimensional arrays with slicing, often a third type of slice specifier exists. This slicing modes takes generally a step or stride parameter in addition to a begin and end value.

For example obtaining a sub-slice starting at the 5th element and taking every 3rd element up to the 12 can be achieved as:

- Matlab: `A(start:step:end)`

- numpy: `A[start:end:step]`
- Fortran: `A[start:end:step]`

However, we propose a slightly different design. Instead of providing begin, end and a step size, we propose to specify begin, extent and a step size.

This approach has one fundamental advantage: one can combine a runtime start value with a compile time extent, and thus directly generate a static extent for the sub-mdspan.

We propose an aggregate type `strided_index_range` to express this slice specification.

We use a struct with named fields instead of a tuple, in order to avoid confusion with the order of the three values.

2.1.1.2.1 extent vs step choice

As stated above, we propose specifying the extent because it makes taking a submdspan with static extent at a runtime offset easier.

One situation where this would be used is in certain linear algebra algorithms, where one steps through a matrix and obtains submatrices of a specified, compile time known, size.

However, remember the layout and accessor types of a sub mdspan, depend on the input mdspan and the slice specifiers. Thus for a generic mdspan, even if one knows the extents of the resulting mdspan the actual type is somewhat cumbersome to determine.

```
// With start:end:step syntax
template<class T, class E, class L, class A>
void foo(mdspan<T,E,L,A> A) {
    using slice_spec_t = pair<int,int>;
    using subm_t =
        decltype(submdspan(declval<slcie_spec_t>(), declval<slice_spec_t>()));
    using static_subm_t = mdspan<T, extents<typename subm_t::index_type, 4,
        4>,
                                typename subm_t::layout_type,
                                typename subm_t::accessor_type>;

    for(int i=0; i<A.extent(0); i)
        for(int j=0; j<A.extent(1); j++) {
            static_subm_t subm = submdspan(A, slice_spec_t(i,i+4),
                slice_spec_t(j,j+4));
            // ...
        }
}

// With the proposed type:

template<class T, class E, class L, class A>
void foo(mdspan<T,E,L,A> A) {
    using slice_spec_t =
```

```

    strided_index_range<int, integral_constant<int,4>,
        integral_constant<int,1>>>;

    for(int i=0; i<A.extent(0); i)
        for(int j=0; j<A.extent(1); j++) {
            auto subm = submdspan(A, slice_spec_t{i}, slice_spec_t{j});
            // ...
        }
}

```

Furthermore, even if one accepts the more complex specification of `subm` it likely that the first variant would require more instructions, since compile time information about extents is not available during the `submdspan` call.

2.1.1.2.2 Template argument deduction

We really want template argument deduction to work for `strided_index_range`. Languages like Fortran, Matlab, and Python have a concise notation for creating the equivalent kind of slice inline. It would be unfortunate if users had to write

```
auto y = submdspan(x, strided_index_range<int, int, int>{0, 10, 3});
```

instead of

```
auto y = submdspan(x, strided_index_range{0, 10, 3});
```

Not having template argument deduction would make it particularly unpleasant to mix compile-time and run-time values. For example, to express the offset and extent as compile-time values and the stride as a run-time value without template argument deduction, users would need to write

```
auto y = submdspan(x, strided_index_range<integral_constant<size_t, 0>,
    integral_constant<size_t, 10>, 3>{{}, {}, 3});
```

Template argument deduction would permit a consistently value-oriented style.

```
auto y = submdspan(x, strided_index_range{integral_constant<size_t, 0>{},
    integral_constant<size_t, 10>{}, 3});
```

This case of compile-time offset and extent and run-time stride would permit optimizations like

- preserving the input `mdspan`'s accessor type (e.g., for aligned access) when the offset is zero; and
- ensuring that the `submdspan` result has a static extent.

2.1.1.2.3 Designated initializers

We would *also* like to permit use of designated initializers with `strided_index_range`. This would let users choose a more verbose, self-documenting style. It would also avoid

any confusion about the order of arguments.

```
auto y = submdspan(x, strided_index_range{.offset=0, .extent=10,
                                           .stride=3});
```

Designated initializers only work for aggregate types. This has implications for template argument deduction. Template argument deduction for aggregates is a C++20 feature. However, few compilers support it currently. GCC added full support in version 11, MSVC in 19.27, and EDG eccp in 6.3, according to the [cppreference.com compiler support table](https://en.cppreference.com/compiler-support-table). For example, Clang 14.0.0 supports designated initializers, and *non*-aggregate (class) template argument deduction, but it does not currently compile either of the following.

```
auto a = strided_index_range{.offset=0, .extent=10, .stride=3};
auto b = strided_index_range<int, int, int>{.offset=0, .extent=10,
                                           .stride=3});
```

Implementers may want to make `mdspan` available for users of earlier C++ versions. The result need not comply fully with the specification, but should be as usable and forward-compatible as possible. Implementers can back-port `strided_index_range` by adding the following two constructors.

```
strided_index_range() = default;
strided_index_range(OffsetType offset_, ExtentType extent_, StrideType
                    stride_)
    : offset(offset_), extent_(extent), stride(stride_)
{}

```

These constructors make `strided_index_range` no longer an aggregate, but restore (class) template argument deduction. They also preserve the struct's properties of being a structural type (usable as a non-type template parameter) and trivially copyable (for compatibility with other programming languages).

§ 2.1.1.3 Compile-time integral values in slices

We also propose that any integral value (on its own, in a tuple, or in `strided_index_range`) can be specified as an `integral_constant`. This ensures that the value can be baked into the return *type* of `submdspan`. For example, layout mappings could be entirely compile time.

Here are some simple examples for rank-1 `mdspan`.

```
int* ptr = ...;
int N = ...;
mdspan a(ptr, N);

// subspace of a single element
auto a_sub1 = submdspan(a, 1);
static_assert(decltype(a_sub1)::rank() == 0);
assert(&a_sub1() == &a(1));
```

```

// subrange
auto a_sub2 = submdspan(a, tuple{1, 4});
static_assert(decltype(a_sub2)::rank() == 1);
assert(&a_sub2(0) == &a(1));
assert(a_sub2.extent(0) == 3);

// subrange with stride
auto a_sub3 = submdspan(a, strided_index_range{1, 7, 2});
static_assert(decltype(a_sub3)::rank() == 1);
assert(&a_sub3(0) == &a(1));
assert(&a_sub3(3) == &a(7));
assert(a_sub3.extent(0) == 4);

// full range
auto a_sub4 = submdspan(a, full_extent);
static_assert(decltype(a_sub4)::rank() == 1);
assert(a_sub4(0) == a(0));
assert(a_sub4.extent(0) == a.extent(0));

```

In multidimensional use cases these specifiers can be mixed and matched.

```

int* ptr = ...;
int N0 = ..., N1 = ..., N2 = ..., N3 = ..., N4 = ...;
mdspan a(ptr, N0, N1, N2, N3, N4);

auto a_sub = submdspan(a, full_extent_t(), 3, strided_index_range{2, N2-5,
    2}, 4, tuple{3, N5-5});

// two integral specifiers so the rank is reduced by 2
static_assert(decltype(a_sub) == 3);
// 1st dimension is taking the whole extent
assert(a_sub.extent(0) == a.extent(0));
// the new 2nd dimension corresponds to the old 3rd dimension
assert(a_sub.extent(1) == (a.extent(2) - 5)/2);
assert(a_sub.stride(1) == a.stride(2)*2);
// the new 3rd dimension corresponds to the old 5th dimension
assert(a_sub.extent(2) == a.extent(4)-8);

assert(&a_sub(1,5,7) == &a(1, 3, 2+5*2, 4, 3+7));

```

§ 2.1.2 Customization Points

In order to create the new `mdspan` from an existing `mdspan` `src`, we need three things:

- the new mapping `sub_map`,
- the new accessor `sub_acc`, and
- the new data handle `sub_handle`.

Computing the new data handle is done via an *offset* and the original accessor's `offset` function, while the new accessor is constructed from the old accessor.

That leaves the construction of the new mapping and the calculation of the *offset* handed to the offset function. Both of those operations depend only on the old mapping and the slice specifiers.

In order to support calling `submdspan` on `mdspan` with custom layout policies, we need to introduce two customization points for computing the mapping and the offset. Both take as input the original mapping, and the slice specifiers.

```
template<class Mapping, class ... SliceArgs>
auto submdspan_mapping(const Mapping&, SliceArgs...) { /* ... */ }

template<class Mapping, class ... SliceArgs>
size_t submdspan_offset(const Mapping&, SliceArgs...) { /* ... */ }
```

Since both of these function may require computing similar information, one should actually fold `submdspan_offset` into `submdspan_mapping` and make that single customization point return a pair of the submapping and offset.

With these components we can sketch out the implementation of `submdspan`.

```
template<class T, class E, class L, class A,
        class ... SliceArgs>
auto submdspan(const mdspan<T,E,L,A>& src, SliceArgs ... args) {
    auto sub_map_offset = submdspan_mapping(src.mapping(), args...);
    typename A::offset_policy sub_acc(src.accessor());
    typename A::offset_policy::data_handle_type
        sub_handle = src.accessor().offset(src.data_handle(), sub_offset);
    return mdspan(sub_handle, sub_map_offset.first, sub_map_offset.second);
}
```

To support custom layouts, `std::submdspan` calls `submdspan_mapping` using argument-dependent lookup.

However, not all layout mappings may support efficient slicing for all possible slice specifier combinations. Thus, we do *not* propose to add this customization point to the layout policy requirements.

§ 2.1.3 Making sure `submdspan` behavior meets expectations

The slice specifiers of `submdspan` completely determine two properties of its result:

1. the extents of the result, and
2. what elements of the input of `submdspan` are also represented in the result.

Both of these things are independent of the layout mapping policy.

The approach we described above orthogonalizes handling of accessors and mappings. Thus, we can define both of the above properties via the multidimensional index spaces, regardless of what it means to “refer to the same element.” (For example, accessors may use proxy references and data handle types other than C++ pointers. This makes it hard to

define what “same element” means.) That will let us define pre-conditions for `submdspan` which specify the required behavior of any user-provided `submdspan_mapping` function.

One function which can help with that, and additionally is needed to implement `submdspan_mapping` for the layouts the standard provides, is a function to compute the `submdspan`'s extents. We will propose this function as a public function in the standard:

```
template<class IndexType, class ... Extents, class ... SliceArgs>
auto submdspan_extents(const extents<IndexType, Extents...>, SliceArgs
    ...);
```

The resulting extents object must have certain properties for logical correctness.

- The rank of the sub-extents is the rank of the original extents minus the number of pure integral arguments in `SliceArgs`.
- The extent of each remaining dimension is well defined by the `SliceArgs`. It is the original extent if all the `SliceArgs` are `full_extent_t`.

For performance and preservation of compile-time knowledge, we also require the following.

- Any `full_extent_t` argument corresponding to a static extent, preserves that static extent.
- Generate a static extent when possible. For example, providing a tuple of `integral_constant` as a slice specifier ensures that the corresponding extent is static.

§ 3 Wording

§ 3.1 Add subsection 22.7.X [mdspan.submdspan] with the following

24.7. submdspan [mdspan.submdspan]

24.7.1 overview [mdspan.submdspan.overview]

- ¹ The submdspan facilities create a new mdspan from an existing input mdspan. The new mdspan's elements refer to a subset of the elements of the input mdspan.

```
namespace std {  
    template<class OffsetType, class LengthType, class StrideType>  
        class strided_index_range;  
  
    template<class IndexType, class... Extents, class... SliceSpecifiers>  
        constexpr auto submdspan_extents(const extents<IndexType,  
            Extents...>&,                               SliceSpecifiers ...);
```

```

template<class E, class... SliceSpecifiers>
constexpr auto submdspan_mapping(
    const layout_left::mapping<E>& src,
    SliceSpecifiers ... slices) -> see below;

template<class E, class... SliceSpecifiers>
constexpr auto submdspan_mapping(
    const layout_right::mapping<E>& src,
    SliceSpecifiers ... slices) -> see below;

template<class E, class... SliceSpecifiers>
constexpr auto submdspan_mapping(
    const layout_stride::mapping<E>& src,
    SliceSpecifiers ... slices) -> see below;

// [mdspan.submdspan], submdspan creation
template<class ElementType, class Extents, class LayoutPolicy,
        class AccessorPolicy, class... SliceSpecifiers>
constexpr auto submdspan(
    const mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy>&
        src,
    SliceSpecifiers...slices) -> see below;
}

```

- 2 The SliceSpecifier template argument(s) and the corresponding value(s) of the arguments of submdspan after src determine the subset of src that the mdspan returned by submdspan views.
- 3 Invocations of submdspan_mapping shown in subclause ([mdspan.submdspan]) select a function call via overload resolution on a candidate set that includes the lookup set found by argument dependent lookup ([basic.lookup.argdep]).
- 4 For each function defined in subsection [mdspan.submdspan] that takes a parameter pack named slices as an argument:

(4.1) — let rank be the number of elements in slices;

(4.2) — let s_k be the k -th element of slices;

(4.3) — let S_k be the type of s_k ; and

(4.4) — let *map-rank* be an array<size_t, rank> such that for each k in the range of $[0, \text{rank})$, *map-rank*[k] equals:

— `dynamic_extent` if `is_convertible_v< $S_k$ , size_t>` is true, or else

— the number of S_j with $j < k$ such that `is_convertible_v< $S_j$ , size_t>` is false.

24.7.2 strided_index_range [mdspan.submdspan.strided_index_range]

```

template<class OffsetType, class ExtentType, class StrideType>
struct strided_index_range {
    using offset_type = OffsetType;

```

```

using extent_type = ExtentType;
using stride_type = StrideType;

OffsetType offset;
ExtentType extent;
StrideType stride;
};

```

1 `strided_index_range` represents a set of extent regularly spaced integer indices. The indices start at `offset`, and increase by increments of `stride`.

2 `strided_index_range` is an aggregate type.

3 *Mandates:*

- `OffsetType`, `ExtentType`, and `StrideType` are signed or unsigned integer types, or are specializations of `integral_constant` that are not a specialization of `bool_constant`.

[Note: `strided_index_range{.offset=1, .extent=10, .stride=3}` indicates the indices 1, 4, 7, and 10. Indices are selected from the half-open interval $[1, 1 + 10)$. - end note]

24.7.3 Exposition-only helpers [mdspan.submdspan.helpers]

```

template<class T>
struct is-strided-index-range: false_type {};

template<class OffsetType, class ExtentType, class StrideType>
struct is-strided-index-range<strided_index_range<OffsetType, ExtentType,
    StrideType>>: true_type {};

```

```

template<class IndexType, class ... SliceSpecifiers>
typename IndexType first_(size_t k, SliceSpecifiers... slices);

```

1 *Mandates:* `IndexType` is a signed or unsigned integer type.

2 Let φ_k denote the following value:

- (2.1) — if `is_convertible_v< $S_k$ , IndexType>` is true, then s_k ;
- (2.2) — otherwise, if `is_convertible_v< $S_k$ , tuple<IndexType, IndexType>>` is true, then `get<0>(  $s_k$  )`;
- (2.3) — otherwise, if `is-strided-index-range< $S_k$ >::value` is true, then s_k .`offset`;
- (2.4) — otherwise, 0.

3 *Preconditions:* φ_k is representable as a value of type `IndexType`.

4 *Returns:* `extents<IndexType>::index-cast(  $\varphi_k$  )`.

```
template<class Extents, class ... SliceSpecifiers>
size_t last_(size_t k, const Extents& ext, SliceSpecifiers... slices);
```

5 *Mandates:* Extents is a specialization of extents.

6 Let `index_type` name the type `typename Extents::index_type`.

7 Let λ_k denote the following value:

(7.1) — if `is_convertible_v< $S_k$ , index_type>` is true, then $s_k + 1$;

(7.2) — otherwise, if `is_convertible_v< $S_k$ , tuple<index_type, index_type>>` is true, then `get<1>(  $s_k$  )`;

(7.3) — otherwise, if `is-strided-index-range< $S_k$ >::value` is true, then $s_k.offset + s_k.extent$;

(7.4) — otherwise, `ext.extent(k)`.

8 *Preconditions:* λ_k is representable as a value of type `index_type`.

9 *Returns:* `Extents::index-cast(  $\lambda_k$  )`.

```
template<class IndexType, size_t N, class ... SliceSpecifiers>
array<IndexType, sizeof...(SliceSpecifiers)> src-indices(const
    array<IndexType, N>& indices, SliceSpecifiers ... slices);
```

10 *Mandates:* `IndexType` is a signed or unsigned integer type.

11 *Returns:* an `array<IndexType, sizeof...(SliceSpecifiers)> src_idx` such that `src_idx[k]` equals

(11.1) — `first_<IndexType>(k, slices...)` for each k where `map-rank[k]` equals `dynamic_extent`, otherwise

(11.2) — `first_<IndexType>(k, slices...) + indices[map-rank[k]]`.

24.7.4 submdspan_extents function [mdspan.submdspan.extents]

```
template<class IndexType, class ... Extents, class ... SliceSpecifiers>
auto submdspan_extents(const extents<IndexType, Extents...>& src_exts,
    SliceSpecifiers ... slices);
```

1 *Constraints:*

(1.1) — `sizeof...(slices)` equals `Extents::rank()`,

(1.2) — For each rank index k of `src_extents()`, at least one of the following is true:

- `is_convertible_v< $S_k$ , IndexType>`,

- `is_convertible_v<Sk, tuple<IndexType, IndexType>>`,
- `is_convertible_v<Sk, full_extent_t>`, and
- `is-strided-index-range<Sk>::value`.

2 *Mandates:* For each rank index k of `src.extents()`, *exactly one* of the following is true:

- (2.1) — `is_convertible_v<Sk, IndexType>`,
- (2.2) — `is_convertible_v<Sk, tuple<IndexType, IndexType>>`,
- (2.3) — `is_convertible_v<Sk, full_extent_t>`, or
- (2.4) — `is-strided-index-range<Sk>::value`.

3 *Preconditions:* For each rank index r of `src.extents()`, *all* of the following are true:

- (3.1) — `0 <= first_<IndexType>(r, slices...)`,
- (3.2) — `first_<IndexType>(r, slices...) <= last_(r, src_exts, slices...)`, and
- (3.3) — `last_(r, src_exts, slices...) <= src_exts.extent(r)`.

4 Let `SubExtents` be a specialization of `extents` such that:

- (4.1) — `SubExtents::rank()` equals the number of k such that `is_convertible_v<Sk, size_t>` is false; and
- (4.2) — for all rank index k of `Extents` such that `map-rank[k] != dynamic_extent` is true, `SubExtents::static_extent(map-rank[k])` equals:
 - `Extents::static_extent(k)` if `is_convertible_v<Sk, full_extent_t>` is true; otherwise
 - `tuple_element<1, Sk>() - tuple_element<0, Sk>()` if S_k is a tuple of two `integral_constant`; otherwise
 - $S_k::\text{extent_type}() / S_k::\text{stride_type}()$ if S_k is a specialization of `strided_index_range`, whose `extent_type` and `stride_type` members are `integral_constant`; otherwise
 - `dynamic_extent`.

5 *Returns:* a value of type `SubExtents` `ext` such that for each k for which `map-rank[k] != dynamic_extent` is true:

- (5.1) — `ext.extent(map-rank[k])` equals $s_k.\text{extent} / s_k.\text{stride}$ if S_k is a specialization of `strided_index_range`, otherwise

- (5.2) — `ext.extent(map-rank[k])` equals `last_(k, src_exts, slices...)` - `first_<IndexType>(k, slices...)`.

24.7.5 Layout specializations of submdspan_mapping

[mdspan.submdspan.mapping]

```
template<class Extents, class... SliceSpecifiers>
constexpr auto submdspan_mapping(
    const layout_left::mapping<Extents>& src,
    SliceSpecifiers ... slices) -> see below;

template<class Extents, class... SliceSpecifiers>
constexpr auto submdspan_mapping(
    const layout_right::mapping<Extents>& src,
    SliceSpecifiers ... slices) -> see below;

template<class Extents, class... SliceSpecifiers>
constexpr auto submdspan_mapping(
    const layout_stride::mapping<Extents>& src,
    SliceSpecifiers ... slices) -> see below;
```

1 Let `index_type` name the type `typename Extents::index_type`.

2 *Constraints:*

- (2.1) — `sizeof...(slices)` equals `Extents::rank()`,
- (2.2) — For each rank index `k` of `src.extents()`, *at least* one of the following is true:
- `is_convertible_v<Sk, index_type>`,
 - `is_convertible_v<Sk, tuple<index_type, index_type>>`,
 - `is_convertible_v<Sk, full_extent_t>`, or
 - `is_strided_index_range<Sk>::value`.

3 *Mandates:* For each rank index `k` of `src.extents()`, *exactly one* of the following is true:

- (3.1) — `is_convertible_v<Sk, index_type>`,
- (3.2) — `is_convertible_v<Sk, tuple<index_type, index_type>>`,
- (3.3) — `is_convertible_v<Sk, full_extent_t>`, or
- (3.4) — `is_strided_index_range<Sk>::value`.

4 *Preconditions:* For each rank index `r` of `src.extents()`, *all* of the following are true:

- (4.1) — `0 <= first_index_type(r, slices...)`,

- (4.2) — *first_<index_type>(r, slices...) <= last_(r, src.extents(), slices...),*
and
- (4.3) — *last_(r, src.extents(), slices...) <= src.extent(r).*
- 5 Let *sub_ext* be the result of *submdspan_extents(src.extents(), slices...)* and let *SubExtents* be *decltype(sub_ext)*.
- 6 Let *sub_strides* be an array<*SubExtents::index_type*, *SubExtents::rank()* such that *sub_strides[map-rank[k]] == src.stride(k)* is true for each rank index *k* of *src.extents()* for which *map-rank[k]* is not *dynamic_extent*.
- 7 Let *P* be a parameter pack such that *is_same_v<make_index_sequence<rank()>, index_sequence<P...>>* is true.
- 8 Let *offset* be a value of type *size_t* equal to *map(first_<index_type>(P, slices...)...)*.
- 9 *Returns:*
- (9.1) — *pair(layout_left::mapping(sub_ext), offset)*, if
- *decltype(src)::layout_type* is *layout_left*; and
 - for each *k* in the range $[0, \text{SubExtents}::\text{rank}() - 1)$, S_k is *full_extent_t*; and
 - for *k* equal to *SubExtents::rank() - 1*, *is_convertible_v<S_k, tuple<index_type, index_type>> || is_convertible_v<S_k, full_extent_t> || is_strided_index_range<S_k>::value* is true; otherwise
- (9.2) — *pair(layout_right::mapping(sub_ext), offset)*, if
- *decltype(src)::layout_type* is *layout_right*; and
 - for each *k* in the range $[\text{Extents}::\text{rank}() - \text{SubExtents}::\text{rank}() + 1, \text{Extents}::\text{rank}())$, S_k is *full_extent_t*; and
 - for *k* equal to $\text{Extents}::\text{rank}() - \text{SubExtents}::\text{rank}()$, *is_convertible_v<S_k, tuple<index_type, index_type>> || is_convertible_v<S_k, full_extent_t> || is_strided_index_range<S_k>::value* is true; otherwise
- (9.3) — *pair(layout_stride::mapping(sub_ext, sub_strides), offset)*.

24.7.6 submdspan function [mdspan.submdspan.submdspan]

```
// [mdspan.submdspan], submdspan creation
template<class ElementType, class Extents, class LayoutPolicy,
         class AccessorPolicy, class... SliceSpecifiers>
constexpr auto submdspan(
```

```
const mdspan<ElementType, Extents, LayoutPolicy, AccessorPolicy>& src,  
SliceSpecifiers...slices) -> see below;
```

1 Let `index_type` name the type `typename Extents::index_type`.

2 *Constraints:*

- (2.1) — `sizeof...(slices)` equals `Extents::rank()`,
- (2.2) — For each rank index `k` of `src.extents()`, *at least* one of the following is true:
 - `is_convertible_v< $S_k$ , index_type>`,
 - `is_convertible_v< $S_k$ , tuple<index_type, index_type>>`,
 - `is_convertible_v< $S_k$ , full_extent_t>`, or
 - *is-strided-index-range*< S_k >::value.
- (2.3) — `submdspan_mapping(src.mapping(), slices...)` is well formed.

3 *Mandates:*

- (3.1) — `is_same_v<decltype(sub_map.extents()),
decltype(submdspan_extents(src.mapping(), slices...))>` is true, and
- (3.2) — For each rank index `k` of `src.extents()`, *exactly one* of the following is true:
 - `is_convertible_v< $S_k$ , index_type>`,
 - `is_convertible_v< $S_k$ , tuple<index_type, index_type>>`,
 - `is_convertible_v< $S_k$ , full_extent_t>`, or
 - *is-strided-index-range*< S_k >::value.

4 *Preconditions:* Let `sub_map_offset` be the result of `submdspan_mapping(src.mapping(), slices...)`. Then:

- (4.1) — For each rank index `r` of `src.extents()`, *all* of the following are true:
 - `0 <= first_<index_type>(r, slices...)`,
 - `first_<index_type>(r, slices...) <= last_(r, src.extents(), slices...)`, and
 - `last_(r, src.extents(), slices...) <= src.extent(r)`;
- (4.2) — `sub_map_offset.first.extents() == submdspan_extents(src.mapping(), slices...)` is true; and

- (4.3) — for each integer pack `I` which is a multidimensional index in `sub_map_offset.first.extents()`, `sub_map_offset.first(I...) + sub_map_offset.second == src.mapping()(src.indices(array{I...}, slices...))` is true.

5 *Effects: Equivalent to*

```
auto sub_map_offset = submdspan_mapping(src.mapping(), args...);
return mdspan(src.accessor().offset(src.data(), sub_map_offset.second),
              sub_map_offset.first,
              AccessorPolicy::offset_policy(src.accessor()));
```

[Example:

Given a rank-3 mdspan `grid3d` representing a three-dimensional grid of regularly spaced points in a rectangular prism, the function `zero_surface` sets all elements on the surface of the 3-dimensional shape to zero. It does so by reusing a function `zero_2d` that takes a rank-2 mdspan.

```
// zero out all elements in an mdspan
template<class T, class E, class L, class A>
void zero_2d(mdspan<T,E,L,A> a) {
    static_assert(a.rank() == 2);
    for(int i=0; i<a.extent(0); i++)
        for(int j=0; j<a.extent(1); j++)
            a[i,j] = 0;
}

// zero out just the surface
template<class T, class E, class L, class A>
void zero_surface(mdspan<T,E,L,A> grid3d) {
    static_assert(grid3d.rank() == 3);
    zero_2d(submdspan(a, 0, full_extent, full_extent));
    zero_2d(submdspan(a, full_extent, 0, full_extent));
    zero_2d(submdspan(a, full_extent, full_extent, 0));
    zero_2d(submdspan(a, a.extent(0)-1, full_extent, full_extent));
    zero_2d(submdspan(a, full_extent, a.extent(1)-1, full_extent));
    zero_2d(submdspan(a, full_extent, full_extent, a.extent(2)-1));
}
```

- end example]